Visual Programming

C# (C sharp)

Graphical User Interfaces with Windows Forms

Lecturer: Dr. Khaled Mohammed BinAbdl

Department of Information Technology,

College of Computers, Seiyun University

Outline

- Introduction
- Windows forms
- Event handling
- Control properties and layout
- Labels, TextBoxes and Buttons
- GroupBoxes and Panels
- CheckBoxes and RadioButtons
- PictureBoxes and ToolTips
- NumericUpDown control
- Mouse event handling and keyboard event handling

Introduction

- A graphical user interface (GUI) allows a user to interact visually with a program.

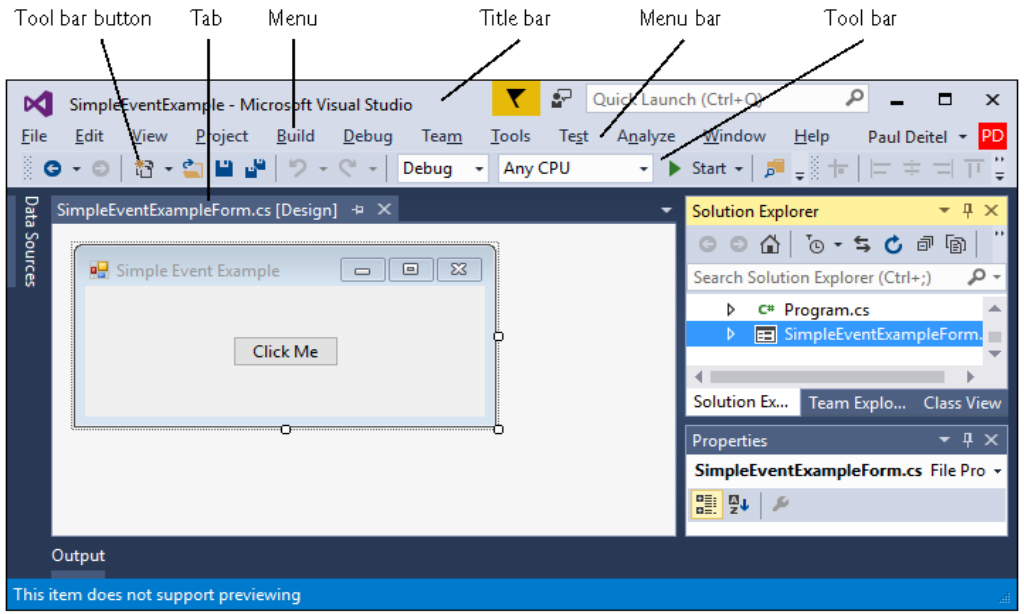


Fig. 14.1 | GUI controls in Visual Studio.

- GUI controls are objects that can display information on the screen or enable users to interact with an app.
- Several common GUI controls are listed in Fig. 14.2.

Control	Description
`Label`	Displays *images* or *uneditable text*.
`TextBox`	Enables the user to *enter data via the keyboard*. It also can be used to *display editable or uneditable text*.
`Button`	Triggers an *event* when clicked with the mouse.
`CheckBox`	Specifies an option that can be *selected* (checked) or *unselected* (not checked).
`ComboBox`	Provides a *drop-down list* of items from which the user can make a *selection* either by clicking an item in the list or by typing in a box.
`ListBox`	Provides a *list* of items from which the user can make a *selection* by clicking one or more items.
`Panel`	A *container* in which controls can be placed and organized.
`NumericUpDown`	Enables the user to select from a *range* of numeric input values.

Fig. 14.2 | Some basic GUI controls.

Windows Forms

- A `Form` is a graphical element that appears on your computer's desktop; it can be a dialog, a window or an MDI window.
- A component is an instance of a class that implements the `IComponent` interface, which defines the behaviors that components must implement, such as how the component is loaded.
- A control, such as a `Button` or `Label`, has a graphical representation at runtime.
- To add a control or component, select it from the **Toolbox** and drag it onto the `Form`.

Windows Forms...

Form properties, methods and an event	Description
Common Properties	
`AcceptButton`	Default `Button` that's clicked when you press *Enter*.
`AutoScroll`	`bool` value (`false` by default) that allows or disallows *scrollbars* when needed.
`CancelButton`	`Button` that's clicked when the *Escape* key is pressed.
`FormBorderStyle`	*Border style* for the `Form` (`Sizable` by default).
`Font`	Font of text displayed on the `Form`, and the *default font* for controls added to the `Form`.
`Text`	Text in the `Form`'s title bar.

Fig. 14.4 | Common `Form` properties, methods and an event. (Part 1 of 2.)

Windows Forms...

Form properties, methods and an event	Description
Common Methods	
`Close`	Closes a `Form` and *releases all resources*, such as the memory used for the `Form`'s contents. A closed `Form` cannot be reopened.
`Hide`	Hides a `Form`, but does *not* destroy the `Form` or release its resources.
`Show`	Displays a *hidden* `Form`.
Common Event	
`Load`	Occurs before a `Form` is displayed to the user. You'll learn about events and event-handling in the next section.

Fig. 14.4 | Common `Form` properties, methods and an event. (Part 2 of 2.)

Event Handling

- GUIs are event driven.
- When the user interacts with a GUI component, the event drives the program to perform a task.
- A method that performs a task in response to an event is called an event handler.

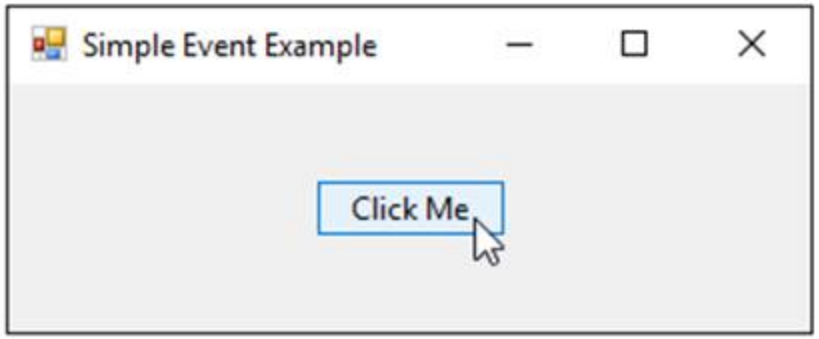
- The `Form` in the app of Fig. 14.5 contains a `Button` that a user can click to display a `MessageBox`.

Event Handling ...

```
// Fig. 14.5: SimpleEventExampleForm.cs
// Simple event handling example.
using System;
using System.Windows.Forms;

namespace SimpleEventExample
{
   // Form that shows a simple event handler
   public partial class SimpleEventExampleForm : Form
   {
      // default constructor
      public SimpleEventExampleForm()
      {
         InitializeComponent();
      }

      // handles click event of Button clickButton
      private void clickButton_Click(object sender, EventArgs e)
      {
         MessageBox.Show("Button was clicked.");
      }
   }
}
```



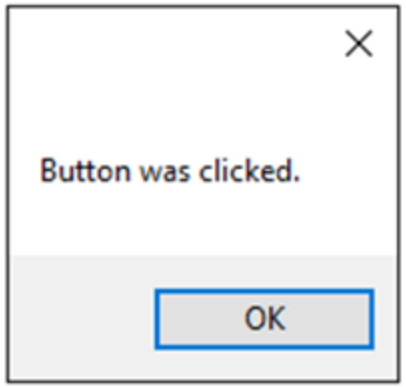


Fig. 14.5 | Simple event-handling example. (Part 1 of 2.)

Event Handling ...

- ***Event Handler Parameters***
- The following empty event handler is declared:
  - ```
    private void clickButton_Click (object sender, EventArgs e)
    {
    }
    ```
- Each event handler receives two parameters when it's called:
  - This first—an object reference typically named sender—is a reference to the object that generated the event.
  - The second is a reference to an `EventArgs` object (or an object of an `EventArgs` derived class) which contains additional information about the event.
    ```

Event Handling ...

Visual Studio places the auto-generated code in the Designer.cs file of the Form.

The partial modifier allows the Form's class to be split among multiple files.

Method InitializeComponent is called when the Form is created.

SimpleEventExampleForm.Designer.cs

SimpleEventExample | SimpleEventExample.SimpleEventExampleForm | components

```
1   namespace SimpleEventExample
2   {
3      partial class SimpleEventExampleForm
4      {
5         /// <summary>
6         /// Required designer variable.
7         /// </summary>
8         private System.ComponentModel.IContainer components = null;
9
10        /// <summary>
11        /// Clean up any resources being used.
12        /// </summary>
13        /// <param name="disposing">true if managed resources should be disposed; oth
14        protected override void Dispose(bool disposing)
15        {
16           if (disposing && (components != null))
17           {
18              components.Dispose();
19           }
20           base.Dispose(disposing);
21        }
22
```

123 %

Fig. 14.6 | First half of the Visual Studio generated code file.

Event Handling ...

Delegates and the Event-Handling Mechanism

- Event handlers are connected to a control's events via special objects called delegates.
- A `delegate` type declaration specifies the signature of a method—in event handling, the signature specifies the return type and arguments for an event handler.
- A `delegate` of type `EventHandler` can hold references to methods that return void and receive two parameters—one of type object and one of type `EventArgs`:
  - `public delegate void EventHandler(object sender, EventArgs e);`

Event Handling ...

Indicating the Method that a Delegate Should Call

- A `Button` calls its `EventHandler` `delegate` in response to a click.
- The `delegate`'s job is to invoke the appropriate method.
- This code is added by Visual Studio when you double click the `Button` control in **Design** mode:
  - `new System.EventHandler(this.clickButton_Click);`
- Adding more `delegates` to an event is called multicast delegates.

Control Properties and Layout

Controls derive from class Control (namespace System.Windows.Forms).

Class Control properties and methods	Description
Common Properties	
BackColor	The control's background color.
BackgroundImage	The control's background image.
Enabled	Specifies whether the control is *enabled* (i.e., if the user can interact with it). Typically, portions of a *disabled* control appear "grayed out" as a visual indication to the user that the control is disabled.
Focused	Indicates whether the control *has the focus* (only available at runtime).
Font	The Font used to display the control's text.
ForeColor	The control's foreground color. This usually determines the color of the text in the Text property.

Fig. 14.11 | Class Control properties and methods. (Part 1 of 3.)

Control Properties and Layout...

Class `Control` properties and methods	Description
`TabIndex`	The *tab order* of the control. When the *Tab* key is pressed, the focus transfers between controls based on the tab order. You can set this order.
`TabStop`	If `true`, then a user can give focus to this control via the *Tab* key.
`Text`	The text associated with the control. The location and appearance of the text vary depending on the type of control.
`Visible`	Indicates whether the control is visible.

Fig. 14.11 | Class `Control` properties and methods. (Part 2 of 3.)

Control Properties and Layout...

Class `Control` properties and methods	Description
Common Methods	
`Hide`	Hides the control (sets the `Visible` property to `false`).
`Select`	*Acquires the focus.*
`Show`	Shows the control (sets the `Visible` property to `true`).

Fig. 14.11 | Class `Control` properties and methods. (Part 3 of 3.)

Control Properties and Layout…

Anchoring and Docking

- Anchoring causes controls to remain at a fixed distance from the sides of the container.
- Anchor a control to the right and bottom sides by setting the `Anchor` property.

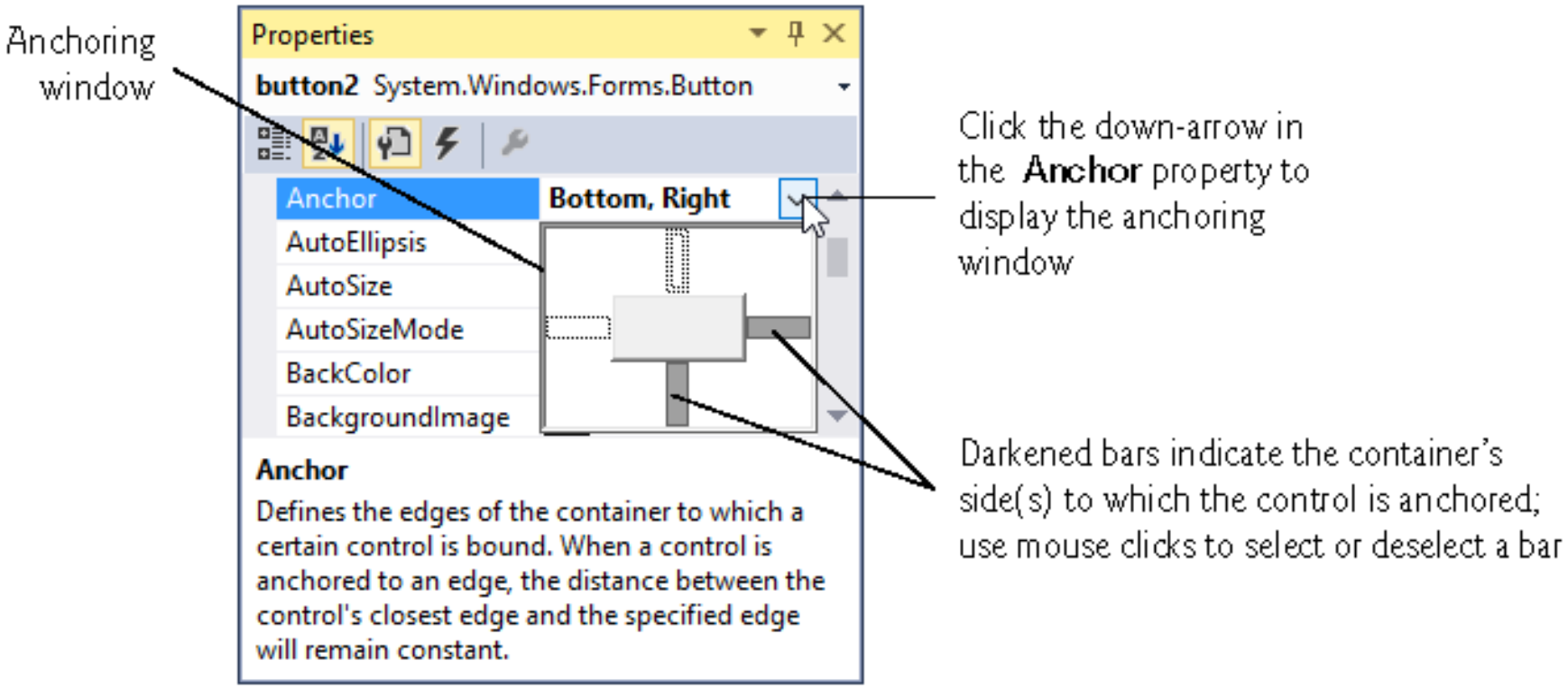


Fig. 14.12 | Manipulating the **Anchor** property of a control.

Control Properties and Layout…

Execute the app and enlarge the Form.

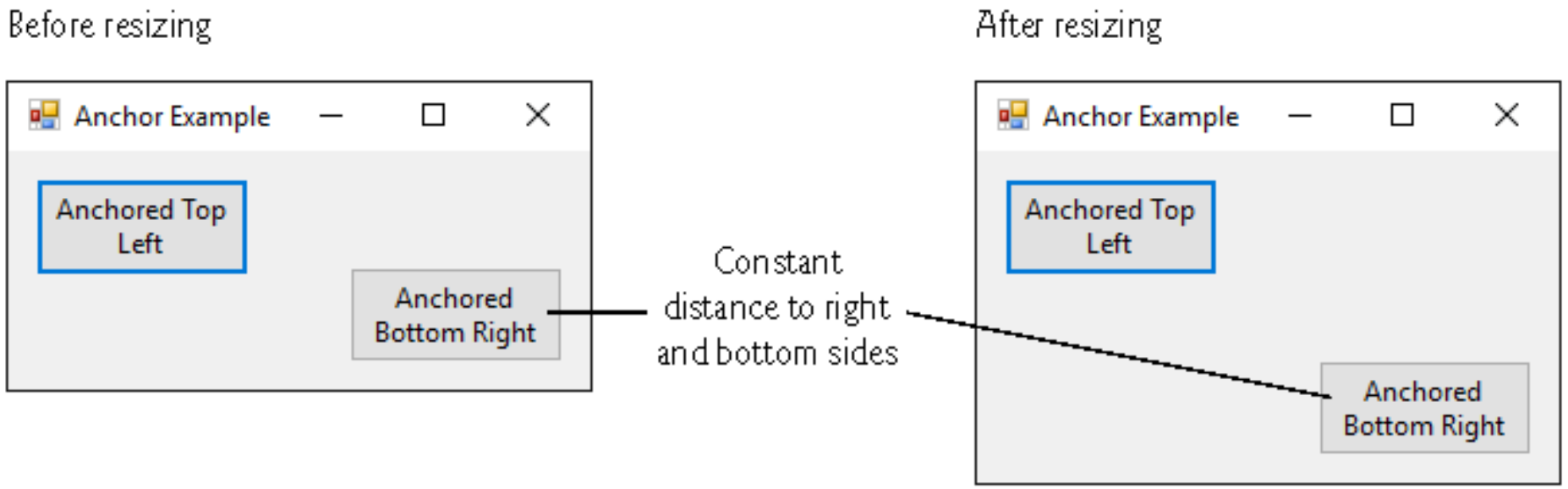


Fig. 14.13 | Anchoring demonstration.

Control Properties and Layout…

- Docking attaches a control to a container such that the control stretches across an entire side or fills an entire area.
- Docking allows a control to span an entire side of its parent container or to fill the entire container.
- The `Form`'s `Padding` property specifies the distance between the docked controls and the edges.

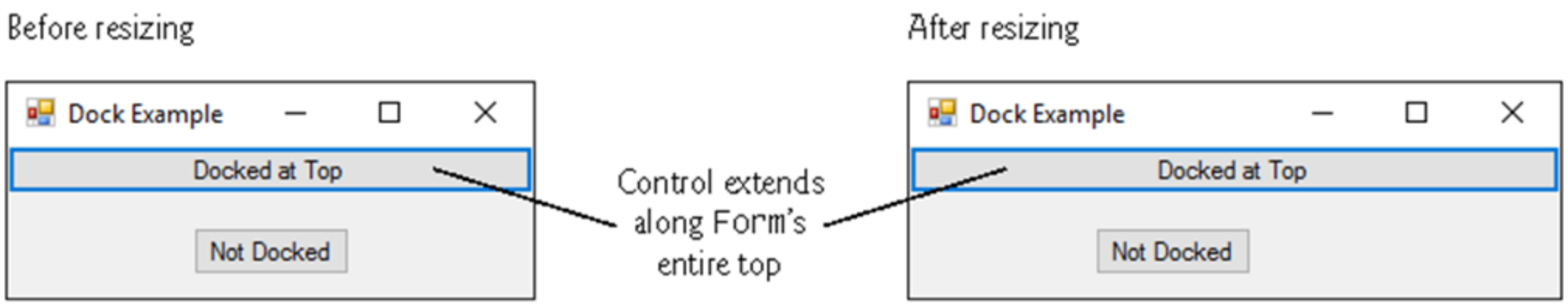


Fig. 14.14 | Docking a **Button** to the top of a **Form**.

`Control` layout properties	Description
`Anchor`	Causes a control to remain at a fixed distance from the side(s) of the container even when the container is resized.
`Dock`	Allows a control to span one side of its container or to fill the remaining space in the container.
`Padding`	Sets the space between a container's edges and docked controls. The default is `0`, causing the control to appear flush with the container's sides.
`Location`	Specifies the location (as a set of coordinates) of the upper-left corner of the control, in relation to its container's upper-left corner.
`Size`	Specifies the size of the control in pixels as a `Size` object, which has properties `Width` and `Height`.
`MinimumSize`, `MaximumSize`	Indicates the minimum and maximum size of a `Control`, respectively.

Fig. 14.15 | `Control` layout properties.

Control Properties and Layout…

- ***Using Visual Studio To Edit a GUI's Layout***
- Visual Studio provides tools that help you with GUI layout.
- When dragging a control across a `Form`, blue lines (known as snap lines) help you position the control.
- Visual Studio also provides the **Format** menu, which contains several options for modifying your GUI's layout.

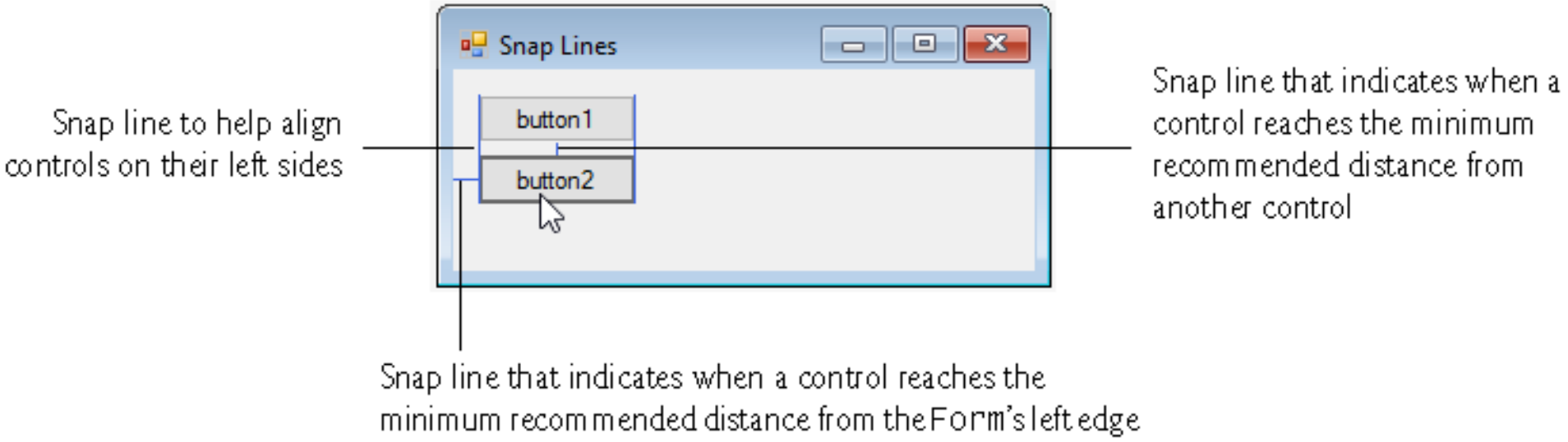


Fig. 14.16 | Snap lines for aligning controls.

Labels, TextBoxes and Buttons

- `Labels` display text that the user cannot directly modify.
- A `TextBox` is an area in which either text can be displayed by a program or the user can type text via the keyboard.
- If you set the property `UseSystemPasswordChar` to `true`, the `TextBox` becomes a password `TextBox`.
- A button is a control that the user clicks to trigger a specific action or to select an option in a program.
- A program can use several types of buttons, but all the button classes derive from class `ButtonBase`.

Look at VS for properties of these controls

Labels, TextBoxes and Buttons...

```
// Fig. 14.20: LabelTextBoxButtonTestForm.cs
// Using a TextBox, Label and Button to display
// the hidden text in a password TextBox.
using System;
using System.Windows.Forms;

namespace LabelTextBoxButtonTest
{
   // Form that creates a password TextBox and
   // a Label to display TextBox contents
   public partial class LabelTextBoxButtonTestForm : Form
   {
      // default constructor
      public LabelTextBoxButtonTestForm()
      {
         InitializeComponent();
      }
```

Fig. 14.20 | Program to display hidden text in a password box.

Labels, TextBoxes and Buttons…

```
18
19          // display user input in Label
20          private void displayPasswordButton_Click(object sender, EventArgs e)
21          {
22             // display the text that the user typed
23             displayPasswordLabel.Text = inputPasswordTextBox.Text;
24          }
25       }
26    }
```

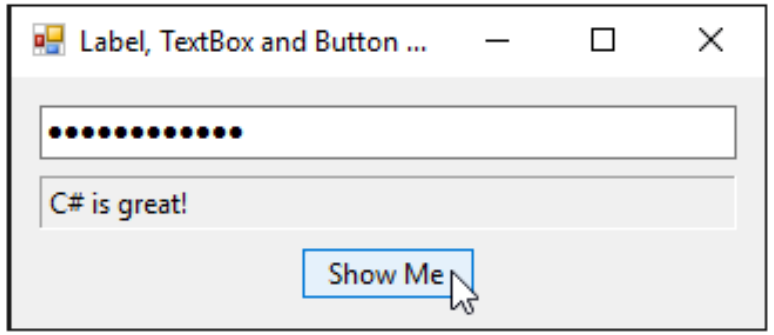


Fig. 14.20 | Program to display hidden text in a password box.

GroupBoxes and Panels

- ***GroupBoxes and Panels arrange related controls on a GUI.***
- All of the controls in a `GroupBox` or `Panel` move together when the `GroupBox` or `Panel` is moved.
- The primary difference is that `GroupBoxes` can display a caption and do not include scrollbars, whereas `Panels` can include scrollbars and do not include a caption.

Look at VS for properties of these controls

GroupBoxes and Panels…

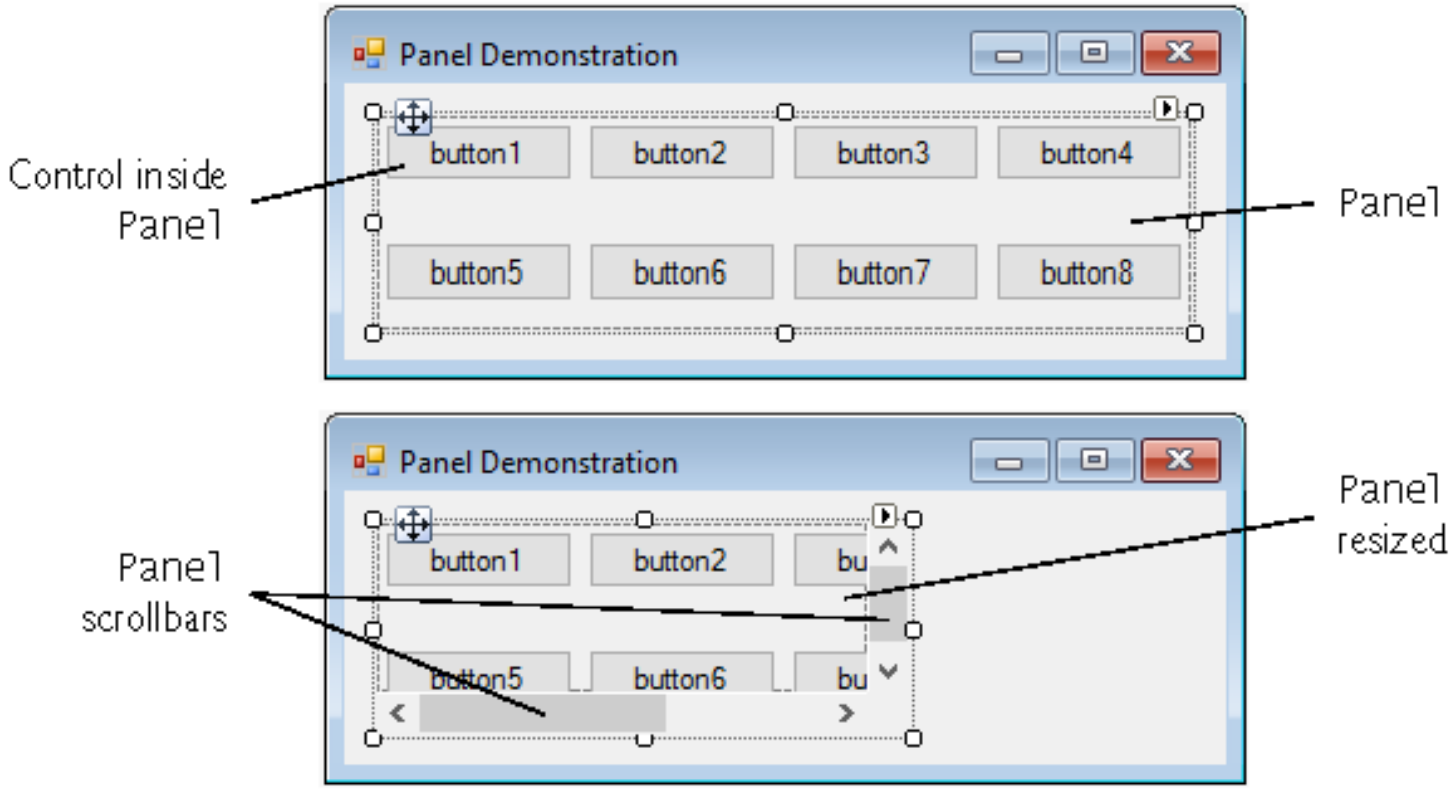


Fig. 14.23 | Creating a **Panel** with scrollbars.

GroupBoxes and Panels...

```
// Fig. 14.24: GroupBoxPanelExampleForm.cs
// Using GroupBoxes and Panels to arrange Buttons.
using System;
using System.Windows.Forms;

namespace GroupBoxPanelExample
{
   // Form that displays a GroupBox and a Panel
   public partial class GroupBoxPanelExampleForm : Form
   {
      // default constructor
      public GroupBoxPanelExampleForm()
      {
         InitializeComponent();
      }

      // event handler for Hi Button
      private void hiButton_Click(object sender, EventArgs e)
      {
         messageLabel.Text = "Hi pressed"; // change text in Label
      }
```

Fig. 14.24 | Using GroupBoxes and Panels to arrange Buttons. (Part 1 of 3.)

GroupBoxes and Panels...

```

        // event handler for Bye Button
        private void byeButton_Click(object sender, EventArgs e)
        {
           messageLabel.Text = "Bye pressed"; // change text in Label
        }

        // event handler for Far Left Button
        private void leftButton_Click(object sender, EventArgs e)
        {
           messageLabel.Text = "Far Left pressed"; // change text in Label
        }

        // event handler for Far Right Button
        private void rightButton_Click(object sender, EventArgs e)
        {
           messageLabel.Text = "Far Right pressed"; // change text in Label
        }
     }
  }
```

Fig. 14.24 | Using `GroupBoxes` and `Panels` to arrange `Buttons`. (Part 2 of 3.)

GroupBoxes and Panels…

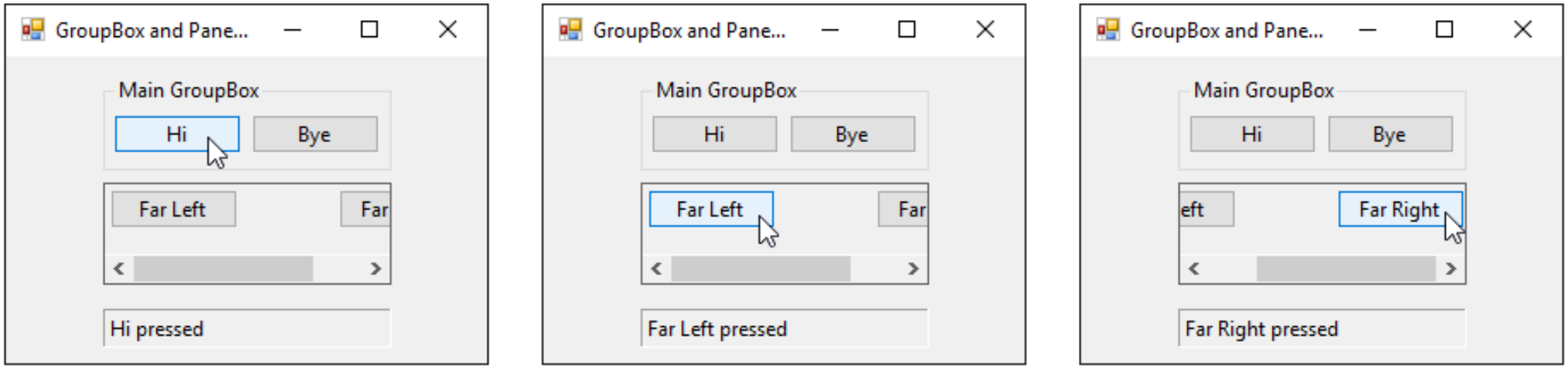


Fig. 14.24 | Using `GroupBox`es and `Panel`s to arrange `Button`s. (Part 3 of 3.)

CheckBoxes and RadioButtons

- ***CheckBoxes***
- A `CheckBox` is a small square that either is blank or contains a check mark.
- You can also configure a `CheckBox` to toggle between three states by setting its `ThreeState` property to `True`.
- Any number of `CheckBoxes` can be selected at a time.

Look at VS for properties of this control

CheckBoxes and RadioButtons...

```
// Fig. 14.26: CheckBoxTestForm.cs
// Using CheckBoxes to toggle italic and bold styles.
using System;
using System.Drawing;
using System.Windows.Forms;

namespace CheckBoxTest
{
   // Form contains CheckBoxes to allow the user to modify sample text
   public partial class CheckBoxTestForm : Form
   {
      // default constructor
      public CheckBoxTestForm()
      {
         InitializeComponent();
      }

```

Fig. 14.26 | Using CheckBoxes to toggle italic and bold styles. (Part 1 of 3.)

CheckBoxes and RadioButtons...

```
        // toggle the font style between bold and
        // not bold based on the current setting
        private void boldCheckBox_CheckedChanged(object sender, EventArgs e)
        {
           outputLabel.Font = new Font(outputLabel.Font,
              outputLabel.Font.Style ^ FontStyle.Bold);
        }

        // toggle the font style between italic and
        // not italic based on the current setting
        private void italicCheckBox_CheckedChanged(
           object sender, EventArgs e)
        {
           outputLabel.Font = new Font(outputLabel.Font,
              outputLabel.Font.Style ^ FontStyle.Italic);
        }
     }
  }
```

Fig. 14.26 | Using CheckBoxes to toggle italic and bold styles. (Part 2 of 3.)

CheckBoxes and RadioButtons…

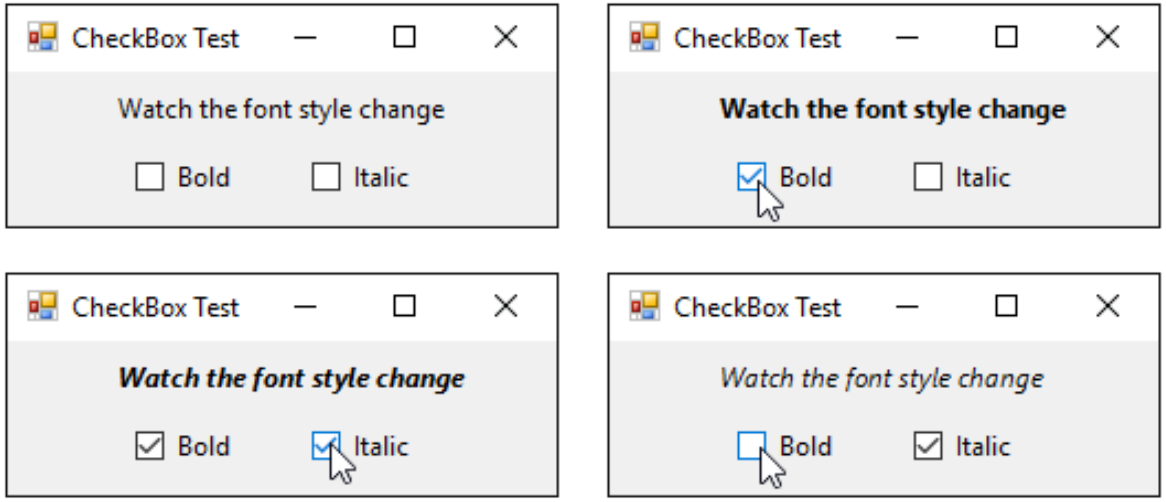


Fig. 14.26 | Using CheckBoxes to toggle italic and bold styles. (Part 3 of 3.)

CheckBoxes and RadioButtons…

- To change the font style on a `Label`, you must set its `Font` property to a new `Font` object.
- The `Font` constructor we used takes the current font and the new style as arguments.
- ***Combining Font Styles with Bitwise Operators***
- Styles can be combined via bitwise operators—operators that perform manipulation on bits of information.
- We needed to set the `FontStyle` so that the text appears in bold if it was not bold originally, and vice versa
  - The logical exclusive OR operator makes toggling the text style simple.

CheckBoxes and RadioButtons (Cont.)

- ***RadioButtons***
- Radio buttons are similar to `CheckBoxes` in that they also have two states—selected and not selected.
- `RadioButtons` normally appear as a group, in which only one `RadioButton` can be selected at a time.
- All `RadioButtons` added to a container become part of the same group.

Look at VS for properties of this control

CheckBoxes and RadioButtons...

```
// Fig. 14.28: RadioButtonsTestForm.cs
// Using RadioButtons to set message window options.
using System;
using System.Windows.Forms;

namespace RadioButtonsTest
{
   // Form contains several RadioButtons--user chooses one
   // from each group to create a custom MessageBox
   public partial class RadioButtonsTestForm : Form
   {
      // create variables that store the user's choice of options
      private MessageBoxIcon IconType { get; set; }
      private MessageBoxButtons ButtonType { get; set; }

      // default constructor
      public RadioButtonsTestForm()
      {
         InitializeComponent();
      }
```

Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 1 of 10.)

CheckBoxes and RadioButtons...

```

        // change Buttons based on option chosen by sender
        private void buttonType_CheckedChanged(object sender, EventArgs e)
        {
            if (sender == okRadioButton) // display OK Button
            {
                ButtonType = MessageBoxButtons.OK;
            }
            // display OK and Cancel Buttons
            else if (sender == okCancelRadioButton)
            {
                ButtonType = MessageBoxButtons.OKCancel;
            }
            // display Abort, Retry and Ignore Buttons
            else if (sender == abortRetryIgnoreRadioButton)
            {
                ButtonType = MessageBoxButtons.AbortRetryIgnore;
            }
```

Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 2 of 10.)

CheckBoxes and RadioButtons...

```
            // display Yes, No and Cancel Buttons
            else if (sender == yesNoCancelRadioButton)
            {
               ButtonType = MessageBoxButtons.YesNoCancel;
            }
            // display Yes and No Buttons
            else if (sender == yesNoRadioButton)
            {
               ButtonType = MessageBoxButtons.YesNo;
            }
            // only one option left--display Retry and Cancel Buttons
            else
            {
               ButtonType = MessageBoxButtons.RetryCancel;
            }
         }

```

Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 3 of 10.)

CheckBoxes and RadioButtons...

```
        // change Icon based on option chosen by sender
        private void iconType_CheckedChanged(object sender, EventArgs e)
        {
           if (sender == asteriskRadioButton) // display asterisk Icon
           {
              IconType = MessageBoxIcon.Asterisk;
           }
           // display error Icon
           else if (sender == errorRadioButton)
           {
              IconType = MessageBoxIcon.Error;
           }
           // display exclamation point Icon
           else if (sender == exclamationRadioButton)
           {
              IconType = MessageBoxIcon.Exclamation;
           }
           // display hand Icon
           else if (sender == handRadioButton)
           {
              IconType = MessageBoxIcon.Hand;
           }
```

Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 4 of 10.)

CheckBoxes and RadioButtons...

```
            // display information Icon
            else if (sender == informationRadioButton)
            {
               IconType = MessageBoxIcon.Information;
            }
            // display question mark Icon
            else if (sender == questionRadioButton)
            {
               IconType = MessageBoxIcon.Question;
            }
            // display stop Icon
            else if (sender == stopRadioButton)
            {
               IconType = MessageBoxIcon.Stop;
            }
            // only one option left--display warning Icon
            else
            {
               IconType = MessageBoxIcon.Warning;
            }
         }
```

Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 5 of 10.)

CheckBoxes and RadioButtons...

```
99
100         // display MessageBox and Button user pressed
101         private void displayButton_Click(object sender, EventArgs e)
102         {
103            // display MessageBox and store
104            // the value of the Button that was pressed
105            DialogResult result = MessageBox.Show(
106               "This is your Custom MessageBox.", "Custon MessageBox",
107               ButtonType, IconType);
108
109            // check to see which Button was pressed in the MessageBox
110            // change text displayed accordingly
111            switch (result)
112            {
113               case DialogResult.OK:
114                  displayLabel.Text = "OK was pressed.";
115                  break;
116               case DialogResult.Cancel:
117                  displayLabel.Text = "Cancel was pressed.";
118                  break;
119               case DialogResult.Abort:
120                  displayLabel.Text = "Abort was pressed.";
121                  break;
```

Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 6 of 10.)

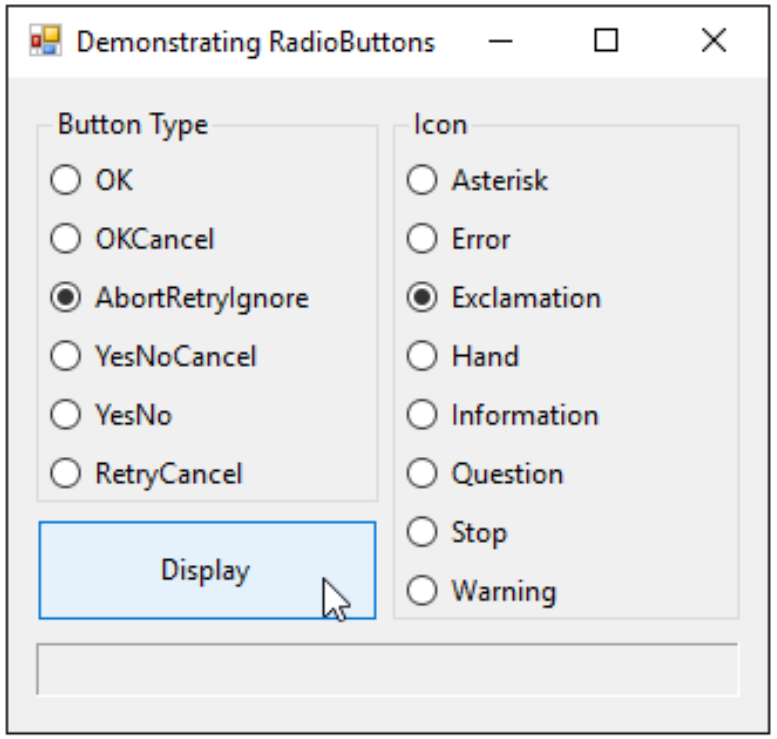
CheckBoxes and RadioButtons...

```
                case DialogResult.Retry:
                   displayLabel.Text = "Retry was pressed.";
                   break;
                case DialogResult.Ignore:
                   displayLabel.Text = "Ignore was pressed.";
                   break;
                case DialogResult.Yes:
                   displayLabel.Text = "Yes was pressed.";
                   break;
                case DialogResult.No:
                   displayLabel.Text = "No was pressed.";
                   break;
             }
          }
       }
    }
```

Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 7 of 10.)

CheckBoxes and RadioButtons…

a) GUI for testing `RadioButtons`



b) `AbortRetryIgnore` button type

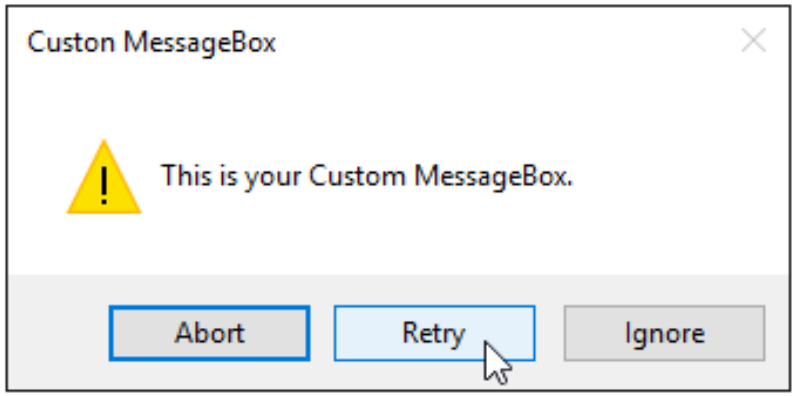
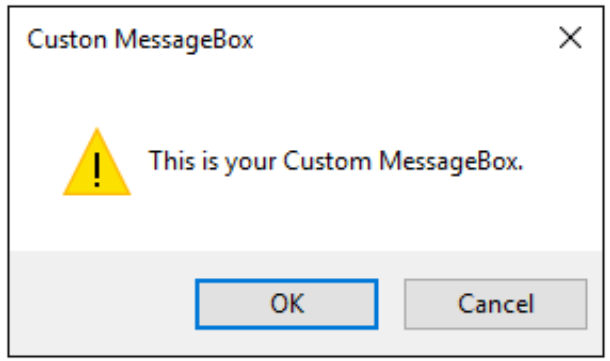


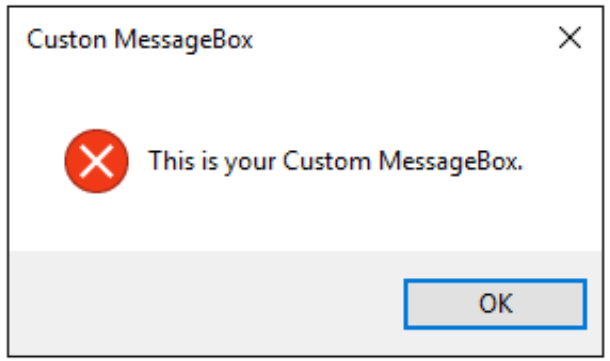
Fig. 14.28 | Using **`RadioButtons`** to set message-window options. (Part 8 of 10.)

CheckBoxes and RadioButtons…

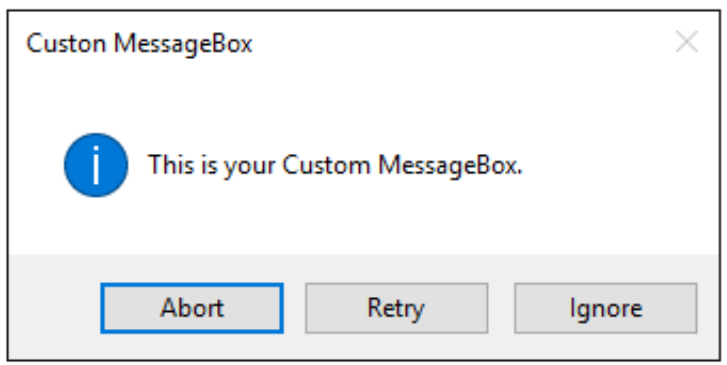
c) `OKCancel` button type



d) `OK` button type



e) `AbortRetryIgnore` button type



f) `YesNoCancel` button type

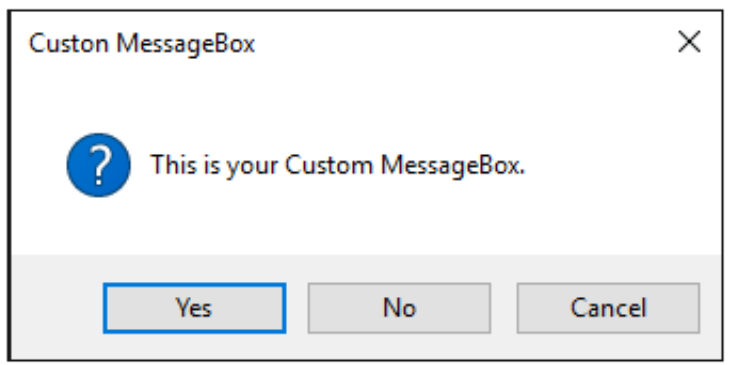
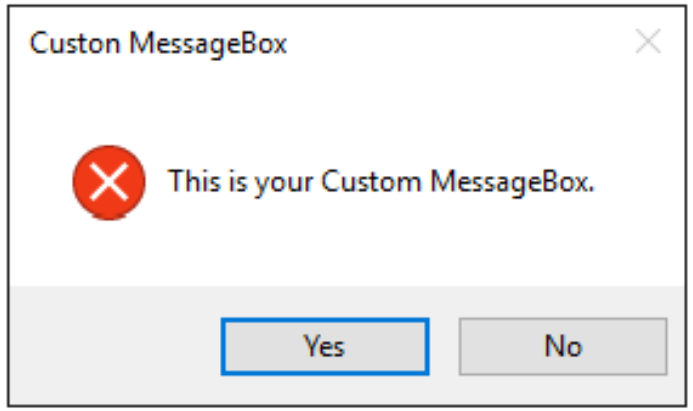


Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 9 of 10.)

CheckBoxes and RadioButtons…

g) YesNo button type



h) RetryCancel button type

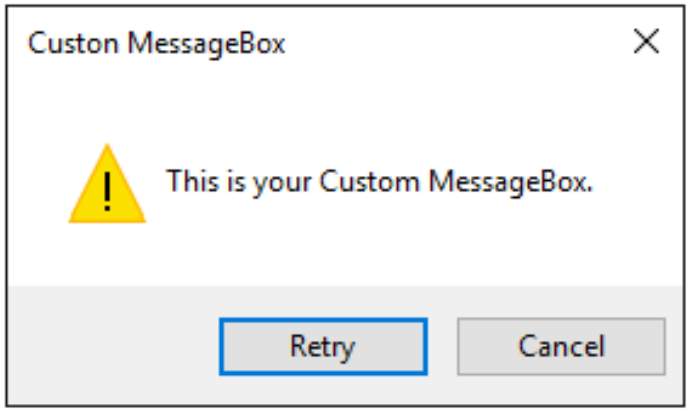


Fig. 14.28 | Using `RadioButtons` to set message-window options. (Part 10 of 10.)

PictureBoxes

A PictureBox displays an image.

Look at VS for properties of this control

```
// Fig. 14.30: PictureBoxTestForm.cs
// Using a PictureBox to display images.
using System;
using System.Drawing;
using System.Windows.Forms;

namespace PictureBoxTest
{
   // Form to display different images when Button is clicked
   public partial class PictureBoxTestForm : Form
   {
      private int ImageNumber { get; set; } = -1; // image to display

      // default constructor
      public PictureBoxTestForm()
      {
         InitializeComponent();
      }
```

Fig. 14.30 | Using a PictureBox to display images. (Part 1 of 3.)

PictureBoxes...

```
        // change image whenever Next Button is clicked
        private void nextButton_Click(object sender, EventArgs e)
        {
            ImageNumber = (ImageNumber + 1) % 3; // cycles from 0 to 2

            // retrieve image from resources and load into PictureBox
            imagePictureBox.Image =
                (Image) (Properties.Resources.ResourceManager.GetObject(
                $"image{ImageNumber}"));
        }
    }
}
```

Fig. 14.30 | Using a `PictureBox` to display images. (Part 2 of 3.)

PictureBoxes…

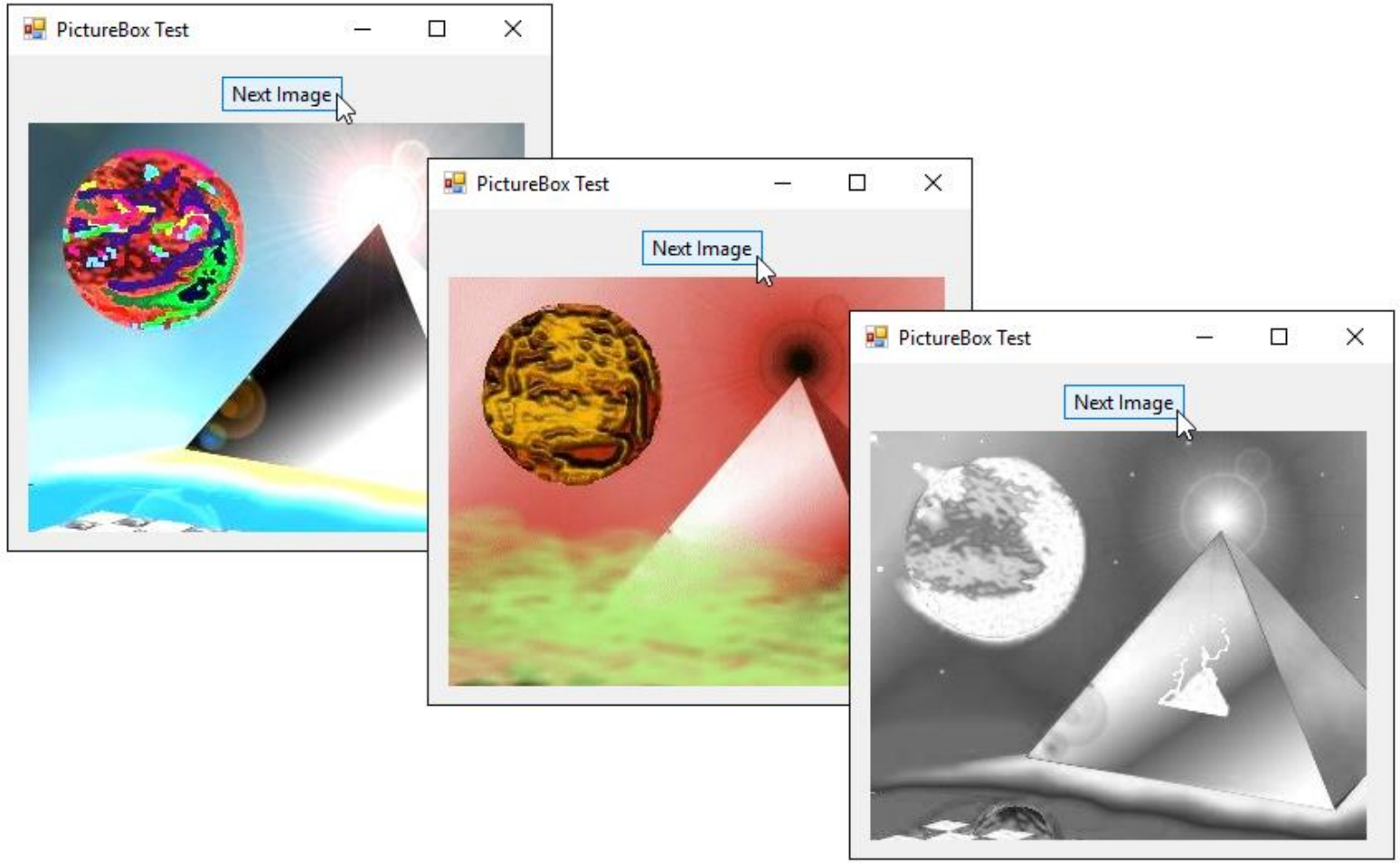


Fig. 14.30 | Using a `PictureBox` to display images. (Part 3 of 3.)

PictureBoxes…

- ***Using Resources Programmatically***
- Embedding the images in the app prevents problems of using several separate files.
- To add a resource:
  - Double click the project's **Properties** node in the **Solution Explorer**.
  - Click the **Resources** tab.
  - At the top of the **Resources** tab click the down arrow next to the **Add Resource** button and select **Add Existing File…**
  - Locate the files you wish to add and click the **Open** button.
  - Save your project.

PictureBoxes…

- A project's resources are stored in its `Resources` class.
- Its `ResourceManager` object allows interacting with the resources programmatically.
- To access an image, use the method `GetObject`, which takes as an argument the resource name as it appears in the **Resources** tab.
- The `Resources` class also provides direct access with expressions of the form `Resources.resourceName`.

ToolTips

- Recall that tool tips are the helpful text that appears when the mouse hovers over an item in a GUI.
- A `ToolTip` component appears in the component tray—the region below the `Form` in **Design** mode.
- A `ToolTip` on property for each `ToolTip` component appears in the **Properties** window for the `Form`'s other controls.

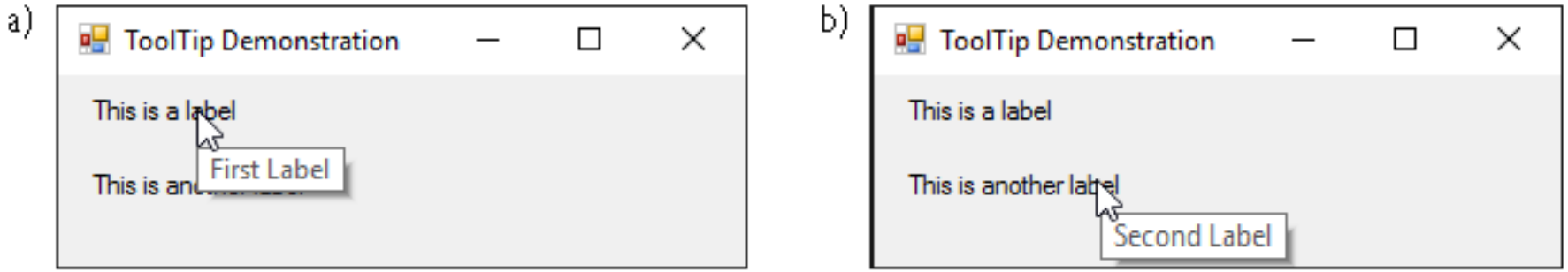


Fig. 14.32 | Demonstrating the `ToolTip` component.

Look at VS for properties of this control

NumericUpDown Control

- Restricting a user's input choices to a specific range of numeric values can be done with a `NumericUpDown` control.
- A user can type numeric values into this control or click up and down arrows.

Look at VS for properties of this control

NumericUpDown Control...

```
// Fig. 14.36: InterestCalculatorForm.cs
// Demonstrating the NumericUpDown control.
using System;
using System.Windows.Forms;

namespace NumericUpDownTest
{
   public partial class InterestCalculatorForm : Form
   {
      // default constructor
      public InterestCalculatorForm()
      {
         InitializeComponent();
      }

```

Fig. 14.36 | Demonstrating the `NumericUpDown` control. (Part 1 of 3.)

NumericUpDown Control...

```
        private void calculateButton_Click(object sender, EventArgs e)
        {
           // retrieve user input
           decimal principal = decimal.Parse(principalTextBox.Text);
           double rate = double.Parse(interestTextBox.Text);
           int year = (int) yearUpDown.Value;

           // set output header
           string output = "Year\tAmount on Deposit\r\n";

           // calculate amount after each year and append to output
           for (int yearCounter = 1; yearCounter <= year; ++yearCounter)
           {
              decimal amount =  principal *
                ((decimal) Math.Pow((1 + rate / 100), yearCounter));
              output += $"{yearCounter}\t{amount:C}\r\n";
           }

           displayTextBox.Text = output; // display result
        }
     }
  }
```

Fig. 14.36 | Demonstrating the `NumericUpDown` control. (Part 2 of 3.)

NumericUpDown Control...

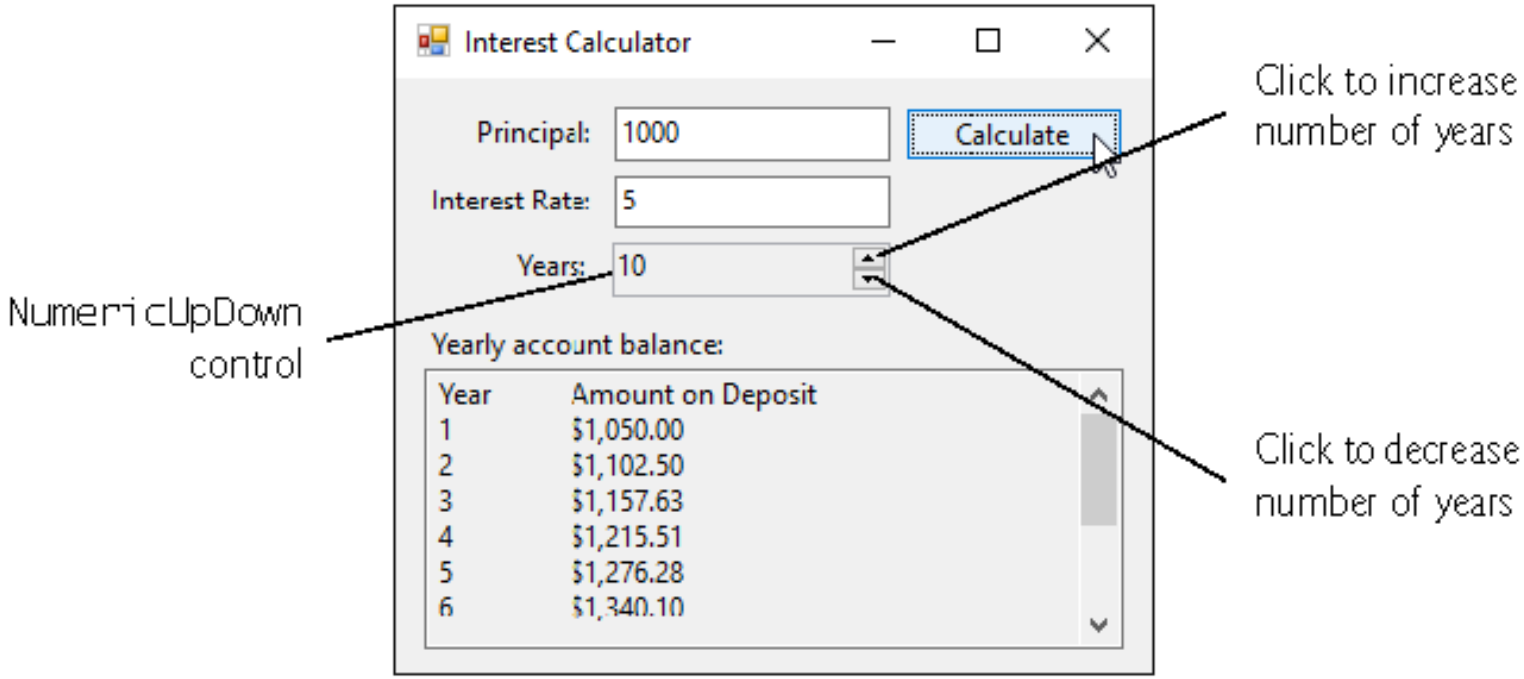


Fig. 14.36 | Demonstrating the `NumericUpDown` control. (Part 3 of 3.)

Mouse-Event Handling

- Mouse events are generated when the user interacts with a control via the mouse.
- Information about the event is passed through a `MouseEventArgs` object, and the delegate type is `MouseEventHandler`.
- `MouseEventArgs` *x*- and *y*-coordinates are relative to the control that generated the event.
- Several common mouse events and event arguments are described in Figure 14.37.

Mouse events and event arguments	
***Mouse Events with Event Argument of Type* `EventArgs`**	
`MouseEnter`	Mouse cursor enters the control's boundaries.
`MouseHover`	Mouse cursor hovers within the control's boundaries.
`MouseLeave`	Mouse cursor leaves the control's boundaries.
***Mouse Events with Event Argument of Type* `MouseEventArgs`**	
`MouseDown`	Mouse button is pressed while the mouse cursor is within a control's boundaries.
`MouseMove`	Mouse cursor is moved while in the control's boundaries.
`MouseUp`	Mouse button is released when the cursor is over the control's boundaries.
`MouseWheel`	Mouse wheel is moved while the control has the focus.

Fig. 14.37 | Mouse events and event arguments. (Part 1 of 2.)

Mouse events and event arguments	
Class* `MouseEventArgs` *Properties	
`Button`	Specifies which mouse button was pressed (`Left`, `Right`, `Middle` or `None`).
`Clicks`	The number of times that the mouse button was clicked.
`X`	The *x*-coordinate within the control where the event occurred.
`Y`	The *y*-coordinate within the control where the event occurred.

Fig. 14.37 | Mouse events and event arguments. (Part 2 of 2.)

Mouse-Event Handling

```
// Fig. 14.38: PainterForm.cs
// Using the mouse to draw on a Form.
using System;
using System.Drawing;
using System.Windows.Forms;

namespace Painter
{
   // creates a Form that's a drawing surface
   public partial class PainterForm : Form
   {
      bool ShouldPaint { get; set; } = false; // whether to paint

      // default constructor
      public PainterForm()
      {
         InitializeComponent();
      }
```

Fig. 14.38 | Using the mouse to draw on a Form. (Part 1 of 4.)

```

        // should paint when mouse button is pressed down
        private void PainterForm_MouseDown(object sender, MouseEventArgs e)
        {
           // indicate that user is dragging the mouse
           ShouldPaint = true;
        }

        // stop painting when mouse button is released
        private void PainterForm_MouseUp(object sender, MouseEventArgs e)
        {
           // indicate that user released the mouse button
           ShouldPaint = false;
        }

```

Fig. 14.38 | Using the mouse to draw on a `Form`. (Part 2 of 4.)

```
        // draw circle whenever mouse moves with its button held down
        private void PainterForm_MouseMove(object sender, MouseEventArgs e)
        {
            if (ShouldPaint) // check if mouse button is being pressed
            {
                // draw a circle where the mouse pointer is present
                using (Graphics graphics = CreateGraphics())
                {
                    graphics.FillEllipse(
                        new SolidBrush(Color.BlueViolet), e.X, e.Y, 4, 4);
                }
            }
        }
    }
}
```

Fig. 14.38 | Using the mouse to draw on a `Form`. (Part 3 of 4.)

Fig. 14.38 | Using the mouse to draw on a `Form`. (Part 4 of 4.)

Mouse-Event Handling (Cont.)

- Recall from Chapter 13 that the using statement automatically calls `Dispose` on the object that was created in the parentheses following keyword using.
- This is important because `Graphics` objects are a limited resource.
- Calling `Dispose` on a `Graphics` object ensures that its resources are returned to the system for reuse.

Keyboard-Event Handling

- There are three key events:
  - The `KeyPress` event occurs when the user presses a key that represents an ASCII character.
  - The `KeyPress` event does not indicate whether modifier keys (e.g., *Shift*, *Alt* and *Ctrl*) were pressed; if this information is important, the `KeyUp` or `KeyDown` events can be used.

Keyboard events and event arguments	
***Key Events with Event Arguments of Type* `KeyEventArgs`**	
`KeyDown`	Generated when a key is initially pressed.
`KeyUp`	Generated when a key is released.
***Key Event with Event Argument of Type* `KeyPressEventArgs`**	
`KeyPress`	Generated when a key is pressed. Raised after `KeyDown` and before `KeyUp`.
Class* `KeyPressEventArgs` *Properties	
`KeyChar`	Returns the ASCII character for the key pressed.
Class* `KeyEventArgs` *Properties	
`Alt`	Indicates whether the *Alt* key was pressed.
`Control`	Indicates whether the *Ctrl* key was pressed.
`Shift`	Indicates whether the *Shift* key was pressed.

Fig. 14.39 | Keyboard events and event arguments. (Part 1 of 2.)

Keyboard events and event arguments	
`KeyCode`	Returns the key code for the key as a value from the `Keys` enumeration. This does not include modifier-key information. It's used to test for a specific key.
`KeyData`	Returns the key code for a key combined with modifier information as a `Keys` value. This property contains all information about the pressed key.
`KeyValue`	Returns the key code as an `int`, rather than as a value from the `Keys` enumeration. This property is used to obtain a numeric representation of the pressed key. The `int` value is known as a Windows virtual key code.
`Modifiers`	Returns a `Keys` value indicating any pressed modifier keys (*Alt*, *Ctrl* and *Shift*). This property is used to determine modifier-key information only.

Fig. 14.39 | Keyboard events and event arguments. (Part 2 of 2.)

```
// Fig. 14.40: KeyDemo.cs
// Displaying information about the key the user pressed.
using System;
using System.Windows.Forms;

namespace KeyDemo
{
   // Form to display key information when key is pressed
   public partial class KeyDemo : Form
   {
      // default constructor
      public KeyDemo()
      {
         InitializeComponent();
      }

      // display the character pressed using KeyChar
      private void KeyDemo_KeyPress(object sender, KeyPressEventArgs e)
      {
         charLabel.Text = $"Key pressed: {e.KeyChar}";
      }
```

Fig. 14.40 | Displaying information about the key the user pressed. (Part 1 of 4.)

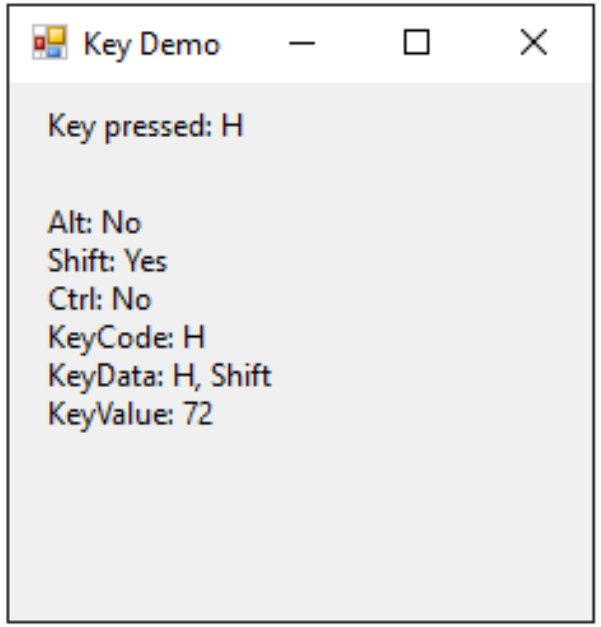
```

        // display modifier keys, key code, key data and key value
        private void KeyDemo_KeyDown(object sender, KeyEventArgs e)
        {
            keyInfoLabel.Text =
                $"Alt: {(e.Alt ? "Yes" : "No")}\n" +
                $"Shift: {(e.Shift ? "Yes" : "No")}\n"  +
                $"Ctrl: {(e.Control ? "Yes" : "No")}\n" +
                $"KeyCode: {e.KeyCode}\n" +
                $"KeyData: {e.KeyData}\n" +
                $"KeyValue: {e.KeyValue}";
        }

        // clear Labels when key released
        private void KeyDemo_KeyUp(object sender, KeyEventArgs e)
        {
            charLabel.Text = "";
            keyInfoLabel.Text = "";
        }
    }
}
```

Fig. 14.40 | Displaying information about the key the user pressed. (Part 2 of 4.)

a) *H* pressed



b) *F7* pressed

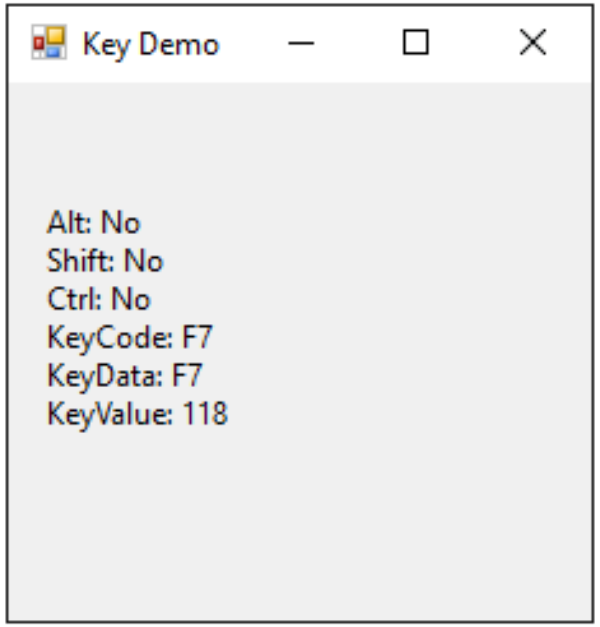
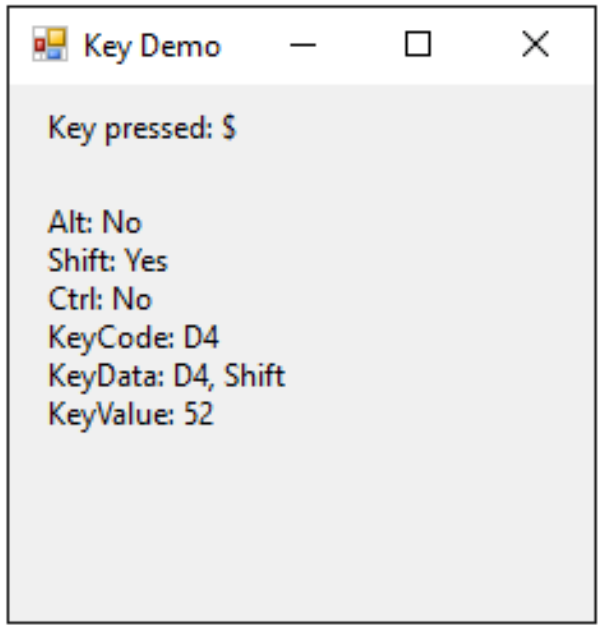


Fig. 14.40 | Displaying information about the key the user pressed. (Part 3 of 4.)

c) $ pressed



d) *Tab* pressed

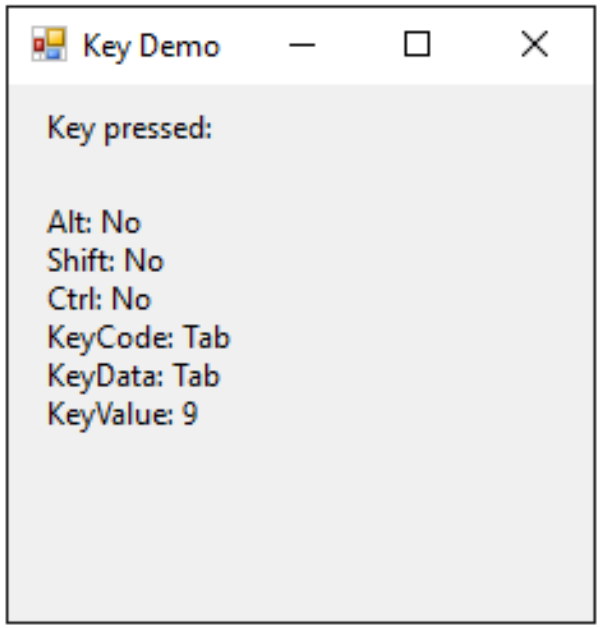


Fig. 14.40 | Displaying information about the key the user pressed. (Part 4 of 4.)